SERVICE ORIENTED ARCHITECTURES

ACIT3855 – WINTER 2026

AGENDA

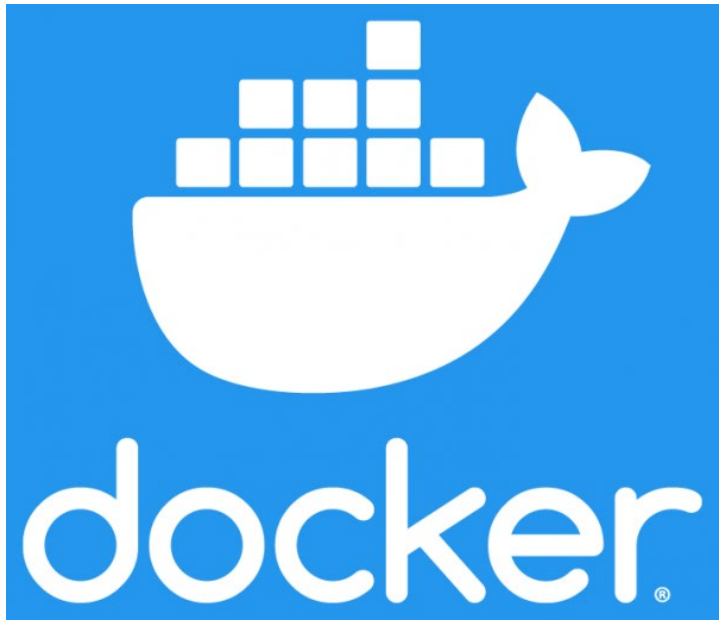
- Midterm
- Quick Review – Docker and Docker Compose, Virtual Environments
- Quiz 7
- Topics:
  - Containerizing Your Services for a Test Environment
- Lab 7 – Demos
- Lab 8 – Containerization with Docker and Docker Compose

MIDTERM - WRITTEN

- How are microservices applications organized?
- How are monolith applications organized?
- When would you and when would you not use an MSA?
- Why do we use externalized configurations in service?
- Why is logging important in a microservices architecture?
- What is the database per service pattern and where do we use it in our services?
- What libraries do we use for synchronous communication and asynchronous communication?

REVIEW

- Docker
- Docker Compose



REVIEW

- Virtual Environments
- requirements.txt

virtualenv

connexion==2.7.0
requests==2.25.1
APscheduler==3.7.0

connexion==2.7.0
pykafka==2.8.0

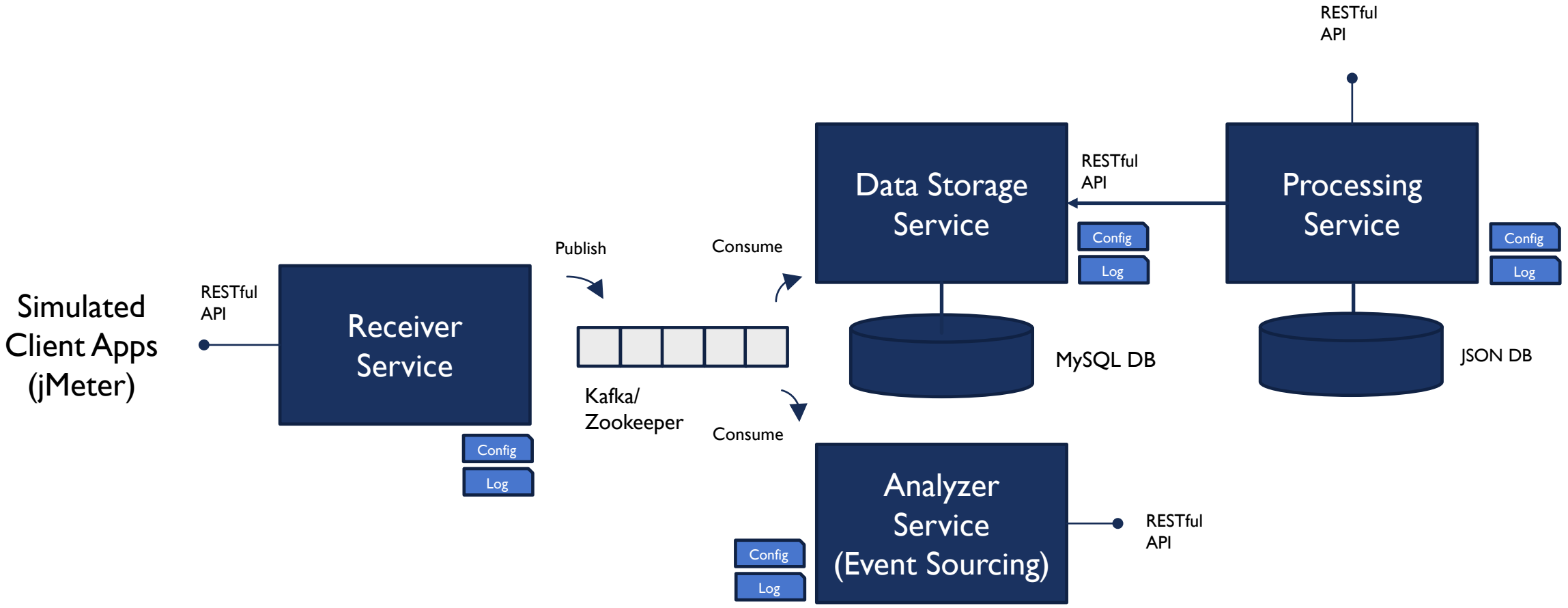
QUIZ 7

- Quiz is on the Learning Hub
- Open book, do your own work
- You have <15 minutes to complete it

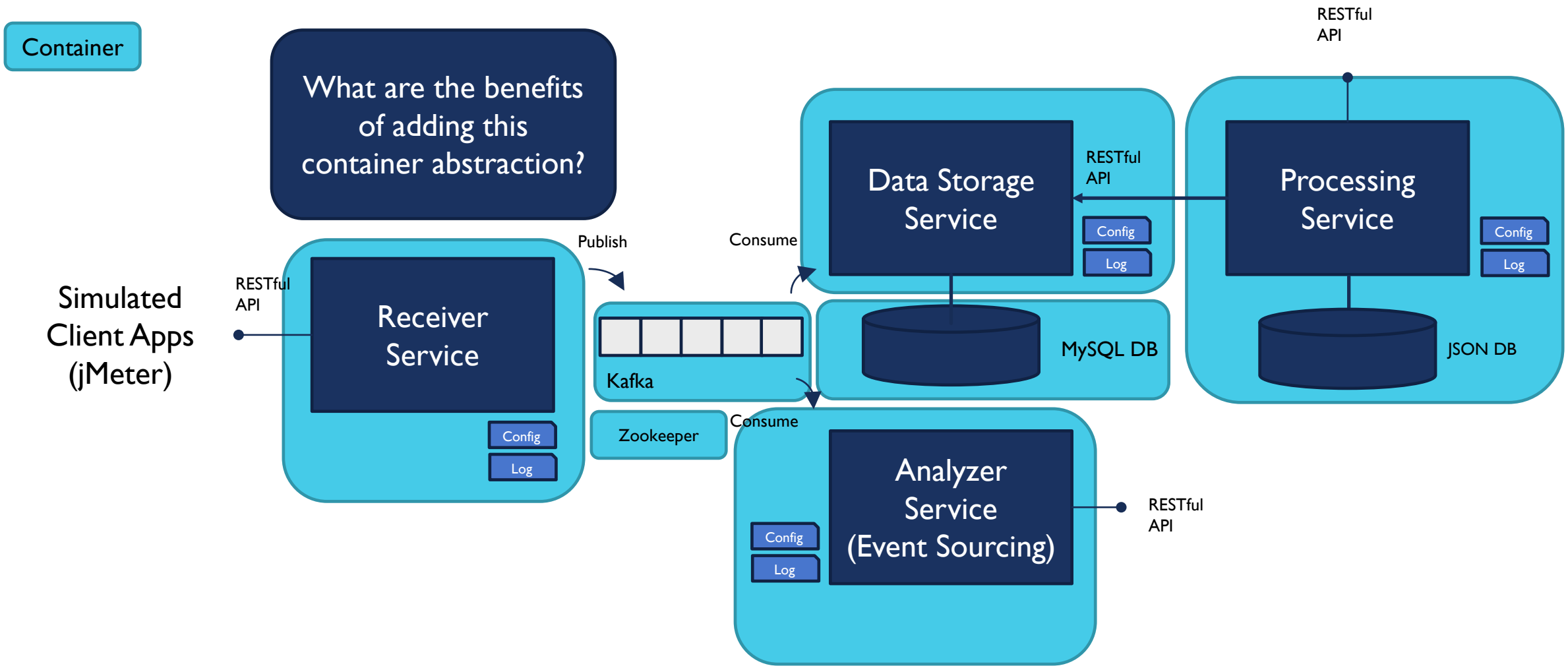
COURSE SCHEDULE

Week	Topics	Notes
1	• Services Based Architecture Overview • RESTful APIs Review	Lab 1
2	• Microservices Overview • Edge Service	Lab 2, Quiz 1
3	• Database Per Service • Storage Service (SQLite)	Lab 3, Quiz 2
4	• Logging, Debugging and Configuration • Storage Service (MySQL)	Lab 4, Quiz 3
5	• RESTful API Specification (OpenAPI) • Processing Service	Lab 5, Quiz 4
6	• Synchronous vs Asynchronous Communication • Message Broker Setup and Messaging	Lab 6, Quiz 5
7	• Event Sourcing • Analysis Service	Lab 7, Quiz 6
8	**Midterm Exam**	
9	• Deployment Part 1 - Containerization (Docker/Docker Compose)	Lab 8, Quiz 7
10	• Deployment Part 2 – Containerization (Docker Volumes and Bind Mounts)	Lab 9, Quiz 8
11	**Computing Reading Week**	
12	• Dashboard UI and CORS	Lab 10, Quiz 9
13	• Deployment Part 3 – Provisioning • Issues and Technical Debt	Lab 11, Quiz 10
14	• Deployment Part 4 – Load Balancing and Scaling • Final Exam Preview	Lab 12, Quiz 11
15	• Review Week	Lab 12 Demos, Assignment Due
16	**Final Exam**	

OUR SAMPLE APPLICATION – LAB 7



OUR SAMPLE APPLICATION – LAB 8 – CONTAINERIZATION

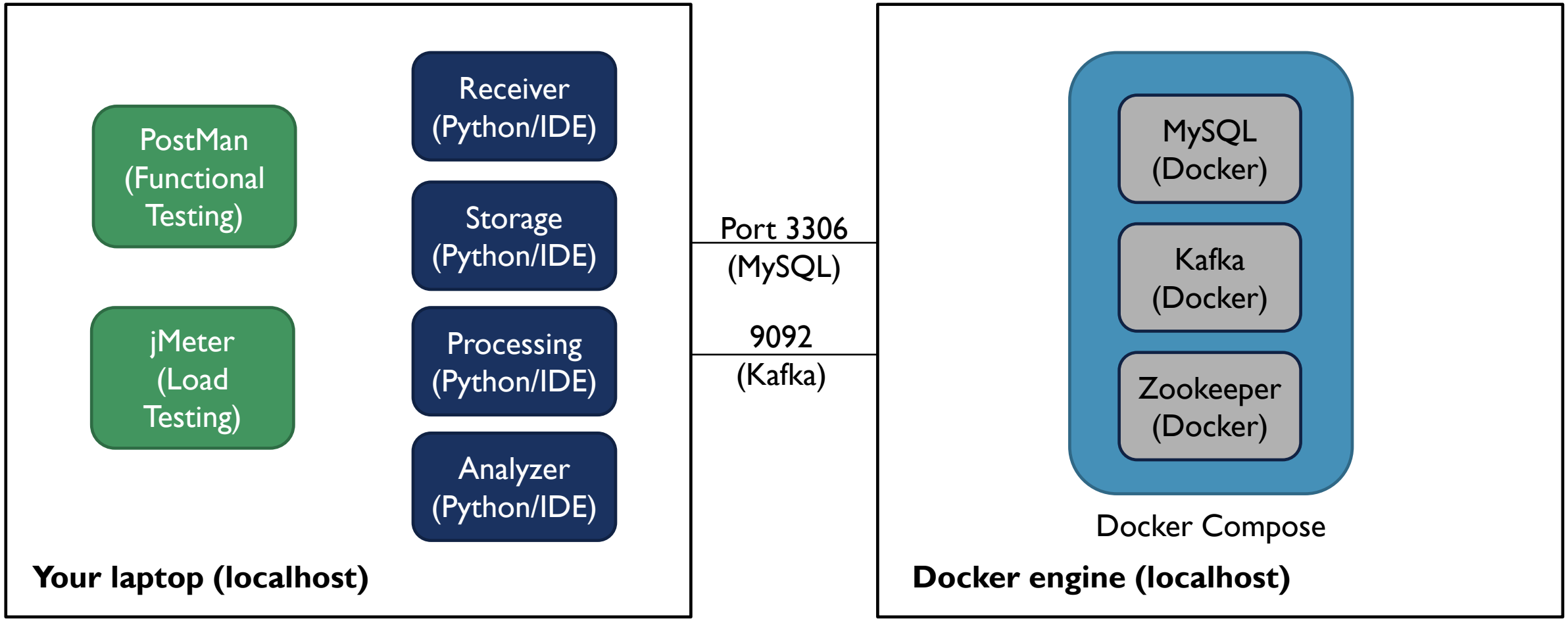


DOCKER - BENEFITS

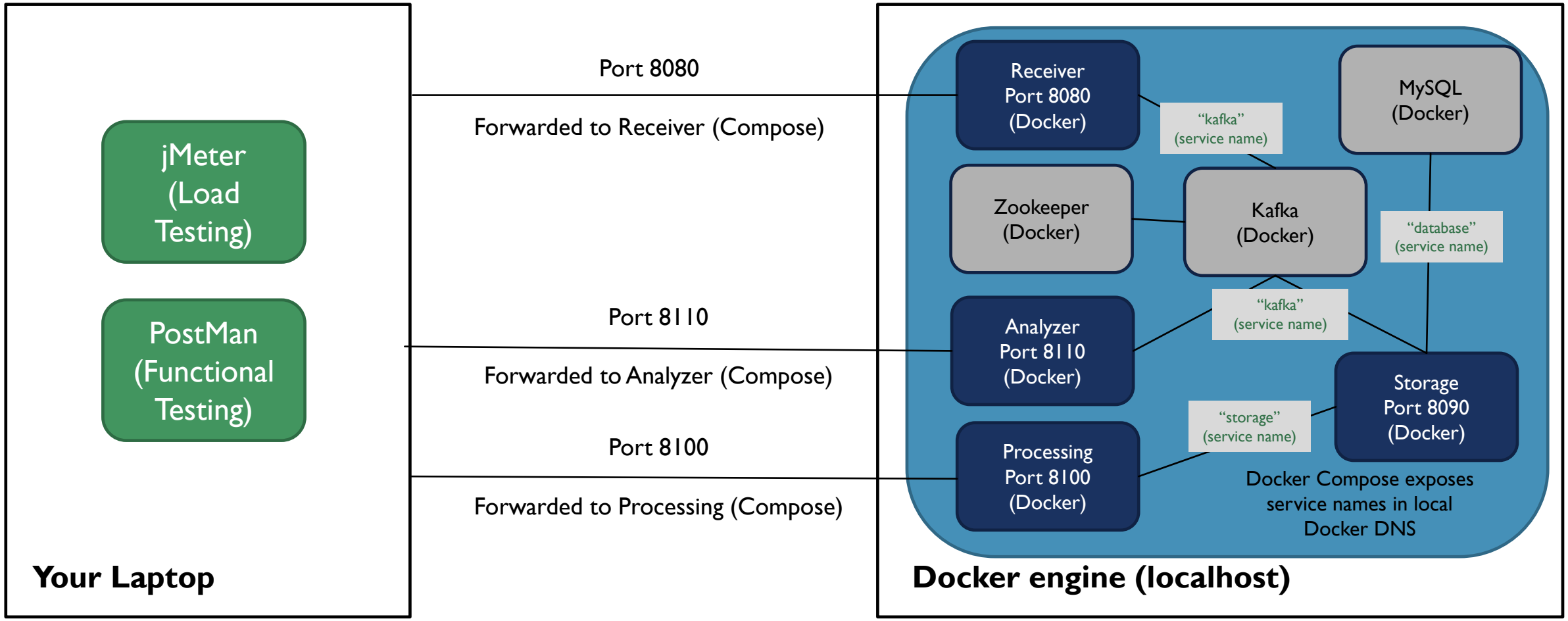
What are some benefits of using Docker?

- Known/standardized configurations – the base image and any additional installed software/configurations are defined in a Dockerfile
- Can start and stop as needed. So can scale up and down as well
- Deployment platforms are built on top of Docker images (i.e., Kubernetes – K8s, OKD)

CURRENT DEVELOPMENT ENVIRONMENT



LAB 8 TEST ENVIRONMENT



DOCKER COMPOSE

Even with Docker, there still are a lot of steps to build and start/stop services when you have many services and/or the services have dependencies (i.e., the order in which the services are started/stopped is important).

Docker Compose provides an abstraction on top of Docker that lets you define service groupings, including 3rd party services (i.e., messaging, DB) in a single file (docker-compose.yml).

Operations then can be applied to the entire service grouping – such as start all (docker-compose up) and stop all (docker-compose down).

Regular Docker commands can still be used even when you use Docker Compose.

DOCKER COMPOSE – RELEVANT COMMANDS

Do not use Docker commands – use their docker-compose alternative!

- docker compose up – to bring all services up
- docker compose up –d – to bring all services up; running in the background
- docker compose up –d --build – to bring all services up; running in the background; will rebuild images
- docker compose down – to bring all services down
- docker compose build – to build services from a docker-compose.yml + Dockerfile
- docker compose ps – to list running services in the current Compose project
- docker compose stop receiver – to stop the "receiver" service container(s)
- docker compose start receiver – to start the "receiver" service container(s)
- docker compose exec receiver bash – to start a shell in the receiver container
- docker compose exec –u 0 receiver bash – to start a shell in the receiver container, as root
- docker compose rm – to remove all stopped containers
- docker compose rm –v – remove all services AND remove their volumes

Here is a Docker Compose cheat sheet of commands: https://devhints.io/docker-compose

DOCKER SUMMARY

Docker	Core for building images and running containers.	Use for a small number of independent services.
Docker Compose	Build on top of Docker and part of the Docker Ecosystem.	Use for groupings of related services where the number and type are static (i.e., no scaling). Good for development and test environments, and simple production environments.
Container Orchestration	Typically 3rd party software for deployment of Docker containers, often over a cluster of VMs. Docker does have the built-in "Docker Swarm".	Use for large number of services and/or when services need to be automatically scaled up and down.

PYTHON 3RD PARTY DEPENDENCIES

- Currently we install them using PIP in our Python environment and/or our IDE
  - It can be hard to keep track of which dependencies you need for a particular project!
- Best practice in Python is to create a requirements.txt file with each 3rd party dependency defined with the version

Example requirements.txt for a Connexion App:

```
connexion==2.7.0
swagger-ui-bundle==0.0.8
pykakfa==2.8.0
```

Run the following to install the dependencies in a test environment:

```
pip3 install -r requirements.txt
```

DOCKER TROUBLESHOOTING

These commands are very helpful when troubleshooting your Docker containers:

- docker compose logs <service name> # Shows the full logs for the service
- docker compose logs <service name> -f # Follows the logs as they are updated
- docker compose logs <service name> --tail=50 -f # Same as above, but only show 50 lines of backlog

DOCKER LOGS

Running Normally

```
azureuser@Acit3855:~/acit3855/receiver$ docker logs c2929e0f0d00
 * Serving Flask app 'app' (lazy loading)
 * Environment: production
   WARNING: This is a development server. Do not use it in a production deployment.
   Use a production WSGI server instead.
 * Debug mode: off
2023-10-19 11:58:02,459 - werkzeug - WARNING -  * Running on all addresses.
   WARNING: This is a development server. Do not use it in a production deployment.
2023-10-19 11:58:02,459 - werkzeug - INFO -  * Running on http://172.17.0.2:8080/ (Press CTRL+C to qui
t)
```

After An Error

```
azureuser@Acit3855:~/acit3855/receiver$ docker logs c2929e0f0d00
 * Serving Flask app 'app' (lazy loading)
 * Environment: production
   WARNING: This is a development server. Do not use it in a production deployment.
   Use a production WSGI server instead.
 * Debug mode: off
2023-10-19 11:58:02,459 - werkzeug - WARNING -  * Running on all addresses.
   WARNING: This is a development server. Do not use it in a production deployment.
2023-10-19 11:58:02,459 - werkzeug - INFO -  * Running on http://172.17.0.2:8080/ (Press CTRL+C to qui
t)
2023-10-19 11:58:52,939 - basicLogger - INFO - Received event Blood Pressure request with a unique id
of d290f1ee-6c54-4b01-90e6-d701748f0851
2023-10-19 11:58:52,960 - app - ERROR - Exception on /blood-pressure [POST]
Traceback (most recent call last):
  File "/usr/local/lib/python3.6/dist-packages/flask/app.py", line 2073, in wsgi_app
    response = self.full_dispatch_request()
  File "/usr/local/lib/python3.6/dist-packages/flask/app.py", line 1518, in full_dispatch_request
    rv = self.handle_user_exception(e)
  File "/usr/local/lib/python3.6/dist-packages/flask/app.py", line 1516, in full_dispatch_request
    rv = self.dispatch_request()
  File "/usr/local/lib/python3.6/dist-packages/flask/app.py", line 1502, in dispatch_request
    return self.ensure_sync(self.view_functions[rule.endpoint])(**req.view_args)
  File "/usr/local/lib/python3.6/dist-packages/connexion/decorators/decorator.py", line 48, in wrapper
    response = function(request)
  File "/usr/local/lib/python3.6/dist-packages/connexion/decorators/uri_parsing.py", line 144, in wrap
per
    response = function(request)
  File "/usr/local/lib/python3.6/dist-packages/connexion/decorators/validation.py", line 184, in wrapp
er
    response = function(request)
  File "/usr/local/lib/python3.6/dist-packages/connexion/decorators/response.py", line 103, in wrapper
    response = function(request)
  File "/usr/local/lib/python3.6/dist-packages/connexion/decorators/parameter.py", line 121, in wrappe
r
    return function(**kwargs)
  File "/app/app.py", line 28, in report_blood_pressure_reading
    client = KafkaClient(hosts=hostname)
  File "/usr/local/lib/python3.6/dist-packages/pykafka/client.py", line 94, in __init__
    ssl_config=ssl_config)
  File "/usr/local/lib/python3.6/dist-packages/pykafka/cluster.py", line 199, in __init__
    self.update()
  File "/usr/local/lib/python3.6/dist-packages/pykafka/cluster.py", line 413, in update
    metadata = self._get_metadata()
  File "/usr/local/lib/python3.6/dist-packages/pykafka/cluster.py", line 276, in _get_metadata
    'Unable to connect to a broker to fetch metadata. See logs.')
pykafka.exceptions.NoBrokersAvailableError: Unable to connect to a broker to fetch metadata. See logs.
2023-10-19 11:58:52,962 - werkzeug - INFO - 192.157.124.16 - - [19/Oct/2023 11:58:52] "POST /blood-pre
ssure HTTP/1.1" 500 -
```

docker logs lets your see the console logs and any stack traces (typically associated with runtime errors). It contains the STDOUT and STDERR output from your application.

This should be your starting point for troubleshooting your services.

DOCKER LABS

- Today's Lab 8:
  - Containerize your services and start/stop all with Docker Compose
  - Due end of next lab
- Next Week's Lab 9:
  - Handle Persistent Data in your services using Docker
  - Centralize Logging and Configurations